

AI Code Assistant

Retrieval-Augmented Generation platform for docs + source code

[Architecture map](#) · [Build status](#) · [Roadmap](#) · [Interview notes](#)

What this document covers

- 1-2 Project vision & complete backend file inventory
- 3-12 Layer-by-layer architecture (ingestion, vectors, reranking, RAG, memory)
- 13-16 Code assistant foundation & the current end-to-end flow
- 17-20 What's done, partial, and not yet built
- 21-24 Advanced code understanding goals, API roadmap, frontend decision
- 25-26 Two known issues to fix before new feature work
- 27-29 Interview talking points & the official project checkpoint

01 What the Project Is

The goal is to build an AI assistant that understands both general technical knowledge (documents) and a user's actual codebase, then answers grounded questions about either — or both together.

```

Upload / register a project
|
Assistant indexes the codebase
|
User asks questions
|
Assistant understands relevant files/code
|
Retrieves relevant code + documentation
|
Reranks the results
|
LLM generates grounded answer
|
Shows sources/files
  
```

Fig. 1 — End-state vision for the assistant.

Example questions the finished assistant should answer:

- “Where is authentication implemented?”
- “Explain the login flow.”
- “Why is this API returning 500?”
- “Which files handle database connections?”
- “Explain this function.”
- “What is the difference between these two classes?”

02 Every Backend File and Its Purpose

This is the most important reference table in the project — every backend file, what it does, and its current build status.

File	Purpose	Status
<code>app.py</code>	Flask API entry point; exposes / and /chat	■ In progress
<code>config.py</code>	Defines supported code extensions and ignored folders	✓ Implemented
<code>database.py</code>	Creates PostgreSQL tables	✓ Implemented
<code>knowledge.py</code>	Small hardcoded knowledge list used for early experiments	■ In progress
<code>pdf_loader.py</code>	Finds PDFs and extracts page-level text	✓ Implemented
<code>chunker.py</code>	Splits text into overlapping chunks	✓ Implemented
<code>hash_utils.py</code>	Generates MD5 hash for files	✓ Implemented
<code>index_registry.py</code>	Tracks which PDFs have already been indexed	✓ Implemented
<code>chunk_registry.py</code>	Stores chunks in <code>all_chunks.json</code> , avoids duplicate chunk IDs	■ In progress
<code>qdrant_db.py</code>	Initializes persistent Qdrant and collection	✓ Implemented
<code>vector_store.py</code>	Creates embeddings and stores vectors in Qdrant	✓ Implemented
<code>retriever.py</code>	Converts query to embedding and retrieves similar vectors	✓ Implemented

File	Purpose	Status
<code>reranker.py</code>	Uses CrossEncoder to rerank retrieved chunks	✓ Implemented
<code>rag_engine.py</code>	Main RAG orchestration: indexing, retrieval, reranking, Gemini generation	■ In progress
<code>conversation_memory.py</code>	PostgreSQL chat/message storage and history retrieval	■ In progress
<code>session_memory.py</code>	Stores long-term session memory in PostgreSQL JSONB	■ In progress
<code>memory_refiner.py</code>	Uses Gemini to refine extracted long-term memory	■ In progress
<code>query_expander.py</code>	Uses Gemini to expand queries for better retrieval	■ In progress
<code>bm25_search.py</code>	Intended BM25 keyword retrieval	✗ Broken
<code>code_loader.py</code>	Recursively reads project source files	■ In progress
<code>project_manager.py</code>	Creates/manages projects in PostgreSQL	■ In progress
<code>test.py</code>	Manually inspects Qdrant records	■ In progress
<code>indexed_files.json</code>	Persistent PDF indexing registry	✓ Implemented
<code>all_chunks.json</code>	Stores chunk information locally	■ In progress
<code>qdrant_data/</code>	Persistent local Qdrant database	✓ Implemented
<code>docs/</code>	Current PDF knowledge base	✓ Implemented

03 Understanding the Files by Layer

Instead of memorizing ~20 files individually, remember the architecture in six layers.

Layer 1 — API (app.py)

The entry point. Currently exposes a single working route.

```
POST /chat
|
generate_answer()
```

Layer 2 — Knowledge Ingestion

These files handle documents, in this order:

pdf_loader.py -> chunker.py -> hash_utils.py -> index_registry.py

pdf_loader.py

PDF -> pages -> page text + source + page number

Example:

```
{
  "source": "DBMS_Notes.pdf",
  "page": 5,
  "text": "..."
```

chunker.py

Turns a large page into smaller pieces. Current configuration: chunk size = 500 characters, overlap = 100 characters. Overlap matters because important information may cross a chunk boundary.

500 chars -> 100-char overlap -> next 500 chars

hash_utils.py

Calculates an MD5 hash for each PDF, used to detect whether a file has already been indexed.

index_registry.py

```
{
  "file_hash": {
    "filename": "DBMS_Notes.pdf"
  }
}
```

On restart:

```
PDF -> hash -> already indexed?
|-- YES -> skip
|-- NO  -> process
```

Prevents unnecessary re-indexing.

Layer 3 — Vector Database

The three most important RAG files: `qdrant_db.py`, `vector_store.py`, `retriever.py`.

qdrant_db.py

Creates the persistent Qdrant store at `qdrant_data/`, collection name `pdf_notes`, vector size 384, distance metric COSINE.

vector_store.py

`chunks -> embedding model -> vectors -> Qdrant`

Metadata stored alongside each vector (this becomes critical for Code RAG later):

- `source`
- `page`
- `text`
- `scope`
- `content_type`
- `chunk_id`
- `indexed_at`

retriever.py

`Question -> Embedding -> Qdrant similarity search -> Top K chunks`

Current RAG pipeline requests: Top 20

Layer 4 — Reranking (reranker.py)

The second retrieval stage — instead of trusting vector search alone:

`Query -> Qdrant -> 20 candidates -> CrossEncoder -> 5 best chunks`

Current model: **cross-encoder/ms-marco-MiniLM-L-6-v2**

"I used a two-stage retrieval architecture. First I used vector similarity to retrieve a broader candidate set, then I used a cross-encoder to rerank those candidates based on query-document relevance."

Layer 5 — RAG Orchestration (rag_engine.py)

Currently the brain of the backend. On startup:

Application starts

|
Find PDFs -> Hash files -> Check indexing registry

|
Extract PDF text -> Chunk text -> Generate embeddings -> Store in Qdrant

On every user query:

User query -> Conversation history -> Retrieval query
-> Qdrant -> Top 20 -> CrossEncoder -> Top 5
-> Context -> Gemini 2.5 Flash -> Answer

This is currently the actual working RAG pipeline.

Layer 6 — Conversation Memory (database.py)

Creates four PostgreSQL tables:

Table	Key Columns
chat_sessions	chat_id, title, created_at
messages	id, chat_id, role, content, timestamp
session_memory	chat_id, memory (JSONB)
projects	project_id, project_name, project_path, status, created_at

conversation_memory.py

Provides create_chat(), add_message(), get_history(), clear_chat(), delete_chat(), get_message_count() — so the assistant can, in theory, use prior turns of a conversation when building the retrieval query.

recent messages + current question -> retrieval query

10 Long-Term Memory (Foundation Only)

Built with `session_memory.py` and `memory_refiner.py`. Intended architecture:

```

Conversation
|
Extract important information
|
Current memory + New memory
|
Gemini refinement
|
Updated memory -> PostgreSQL JSONB

{
  "project": "AI Code Assistant",
  "backend": "Flask",
  "vector_db": "Qdrant",
  "database": "PostgreSQL"
}

```

Example memory object.

Important: the code exists, but the actual extraction/refinement calls in `rag_engine.py` are commented out. Call this a **long-term memory foundation**, not a completed feature.

11 Query Expansion (Disabled)

`query_expander.py` exists and is intended to work like this:

```

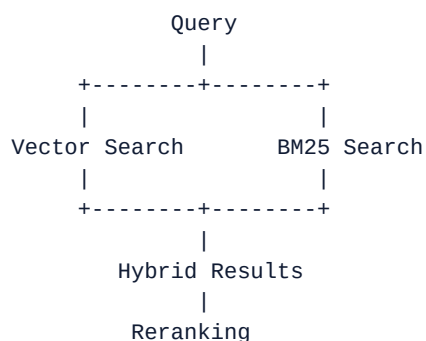
User: "How does JWT authentication work?"
|
Expanded query:
  JWT authentication, token generation, access token,
  refresh token, authorization, middleware ...

```

Currently this section is commented out — query expansion exists as an experiment but is not part of the active RAG pipeline.

12 BM25 (Broken / Incomplete)

`bm25_search.py` was intended to add keyword retrieval alongside vector search:



But the current file calls **BM25Okapi(tokenized_chunks)** without a defined `tokenized_chunks` — hybrid search is not implemented.

13 Code Assistant Foundation

This is where the actual AI Code Assistant begins.

config.py

Defines supported source extensions and ignored folders — so the RAG system never indexes noise like `node_modules` or `.git`.

Supported extensions	Ignored folders
<code>.py .java .cpp .c .js .ts .jsx .tsx</code>	<code>venv .git node_modules __pycache__ .idea .vscode</code> <code>dist build</code>

project_manager.py

Creates a project (`project_name`, `project_path`, `project_id`) in PostgreSQL. Supported functions:

- `create_project()`
- `get_project()`
- `list_projects()`
- `update_project_status()`
- `delete_project()`

code_loader.py — the beginning of Code RAG

```
Project directory
|
os.walk()
|
Check extension -> Ignore unwanted folders -> Read source code
|
Return files

{
  "project_id": "...",
  "file_path": "backend/auth/login.py",
  "extension": ".py",
  "content": "..."
}
```

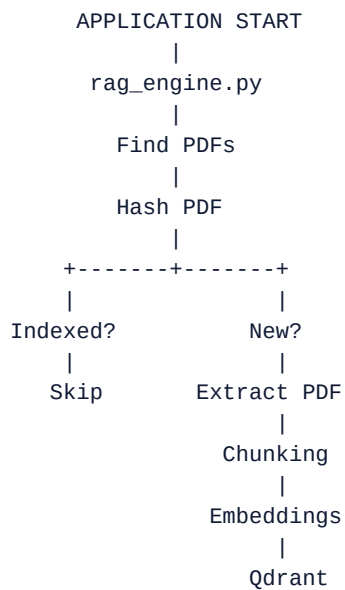
Example output of code_loader.py.

This is only loading. Not yet completed: code -> code-aware chunks -> embeddings -> Qdrant -> code retrieval.

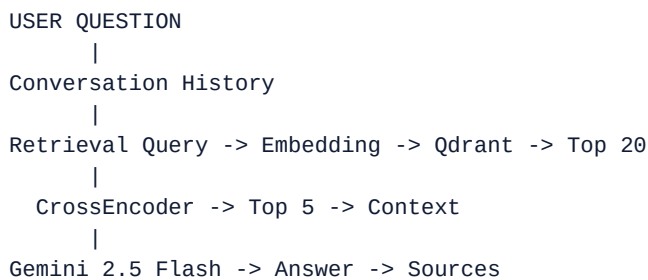
16 Current Complete Flow

Putting it all together — the PDF RAG pipeline as it runs today.

Startup — indexing



Runtime — answering a question



17 What's Working Now

RAG

- PDF ingestion
- Page-level extraction
- Chunking + overlap
- File hashing
- Duplicate indexing prevention
- Embedding generation
- Qdrant storage
- Vector + Top-K retrieval
- Cross-encoder reranking
- Gemini generation
- Source/page metadata
- Conversational retrieval query

Backend & DB

- Flask /, /chat
- CORS + error handling
- PostgreSQL connection
- Chat sessions & messages tables
- Session memory table
- Projects table

Code Foundation

- Project registration / listing / status / deletion
- Supported source-file detection
- Recursive project scanning
- Code file loading

18 What's Partially Implemented

These are dangerous to call “done.”

Feature	Reality
Conversation memory	Backend functions exist, API integration incomplete
Long-term memory	Database + logic exist, workflow disabled
Query expansion	Implemented but disabled
Project management	Backend functions exist, no API endpoints
Code ingestion	File loader exists, no indexing
Chunk registry	Exists but not central to active flow

19 What's Not Implemented Yet

This is the major future work.

Code RAG

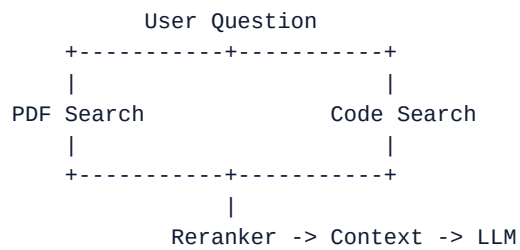
Code files -> Code-aware chunking -> Code embeddings
-> Qdrant -> Project filtering -> Code retrieval

Project-Aware Retrieval

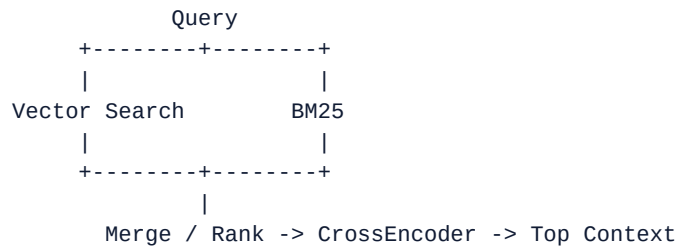
Needs project_id, scope, content_type, file_path, language inside Qdrant metadata, then:

Question -> project_id = X -> retrieve only project X

Unified RAG (PDF + Code)



20 Hybrid Retrieval (Not Done)



21 Advanced Code Understanding

This is what will make the project genuinely an AI Code Assistant rather than a PDF chatbot. Six target capabilities:

- File-level understanding — “Which files implement authentication?”
- Function-level understanding — “Explain this function.”
- Dependency understanding — “What calls this function?”
- Cross-file reasoning — “Explain the login flow from frontend to database.”
- Debugging — “Why might this API return 500?”
- Architecture understanding — “Explain the architecture of this project.”

These require more than simple PDF-style chunking.

22 API Layer Still Needs to Grow

Today, POST /chat is basically the whole API. The target surface:

```

POST   /projects
GET    /projects
GET    /projects/{id}
DELETE /projects/{id}

POST   /projects/{id}/index

POST   /chat

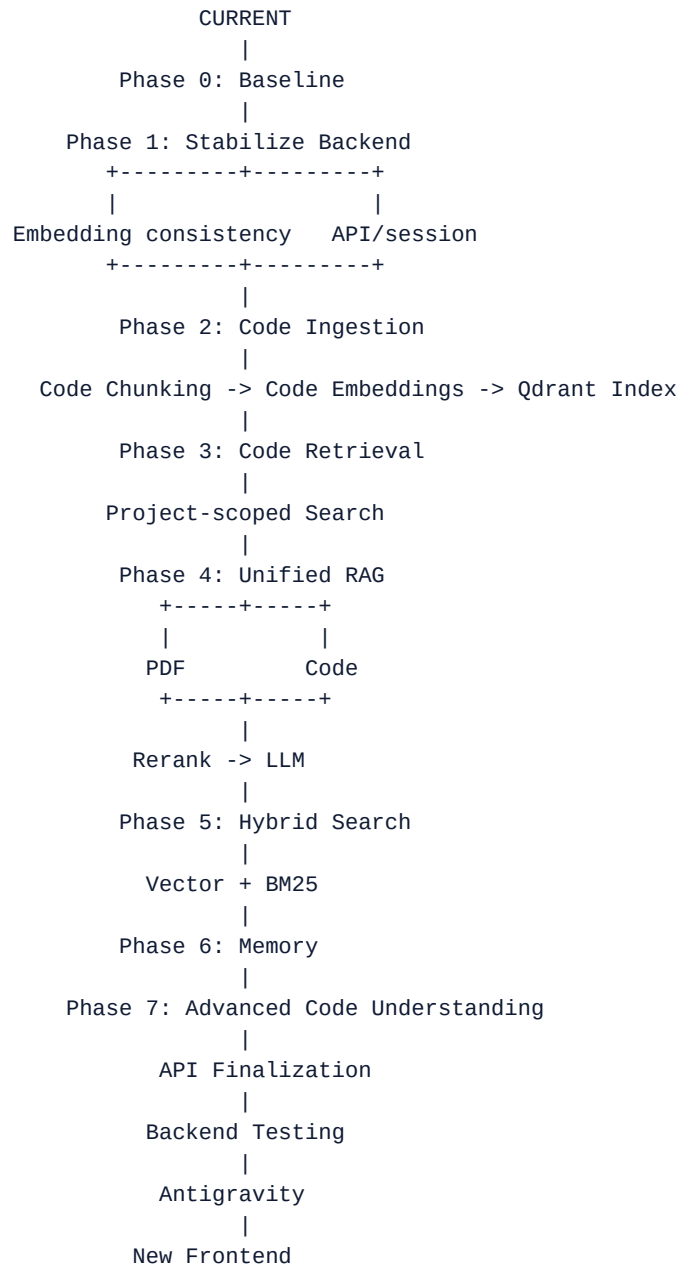
GET    /chats
GET    /chats/{id}
DELETE /chats/{id}
  
```

Exact endpoints can be finalized later.

23 Frontend Decision

Decision: do not continue developing the current frontend. It will be kept as an archive/reference and rebuilt after the backend is stable, using Antigravity. This is useful because the backend API gets designed first, instead of being forced to fit an old UI.

24 The Roadmap From Here



25 Two Issues Before Phase 2

We should not build more features on top of broken foundations. These two must be fixed first.

Issue 1 — Embedding Mismatch

```
vector_store.py -> BAAI/bge-small-en-v1.5
retriever.py    -> all-MiniLM-L6-v2
Qdrant          -> 384 dimensions
```

General rule: **document embedding model = query embedding model**. The vector dimensions and semantic space must match, or retrieval quality silently degrades.

Issue 2 — Backend Chat API Mismatch

```
rag_engine.py expects: generate_answer(query, chat_id)
app.py currently calls: generate_answer(query)
```

Frontend currently sends only: { "message": "..."} (no chat_id)

The proper backend contract needs to be designed before continuing.

26 Database Architecture Note

Current files create long-lived global PostgreSQL connections: database.py, conversation_memory.py, session_memory.py, project_manager.py. For a production-quality backend, a cleaner connection/pool strategy is needed eventually — it doesn't need to be redesigned immediately, but it belongs in the backend-stabilization phase.

27 What to Remember for Interviews

Don't undersell it as "a chatbot using Gemini." Use language like this instead:

"I am building an AI Code Assistant using a RAG architecture. The system is designed to ingest both technical documentation and a user's source code, convert the content into embeddings, store it in Qdrant, retrieve relevant context using vector search, rerank the retrieved candidates using a cross-encoder, and then use an LLM to generate a grounded response. I'm also building project-level isolation and conversational memory so the assistant can reason about a specific codebase across multiple queries."

Why Qdrant?

Because we need semantic similarity search over document and code chunks. Qdrant provides vector storage and efficient similarity retrieval, and lets us attach metadata such as project ID, file path, content type, and page information.

Why reranking?

Vector search gives a broad candidate set, but similarity alone isn't always enough to determine exact relevance. I retrieve a larger set first, then use a CrossEncoder to rerank before sending the best context to the LLM.

Why chunking?

We can't send an entire document or codebase to the LLM. Chunking creates smaller retrievable units and lets the system select only the relevant context.

Why overlap?

To avoid losing information that crosses a chunk boundary.

Why hashing?

To avoid repeatedly embedding files that haven't changed.

Retrieval vs. reranking?

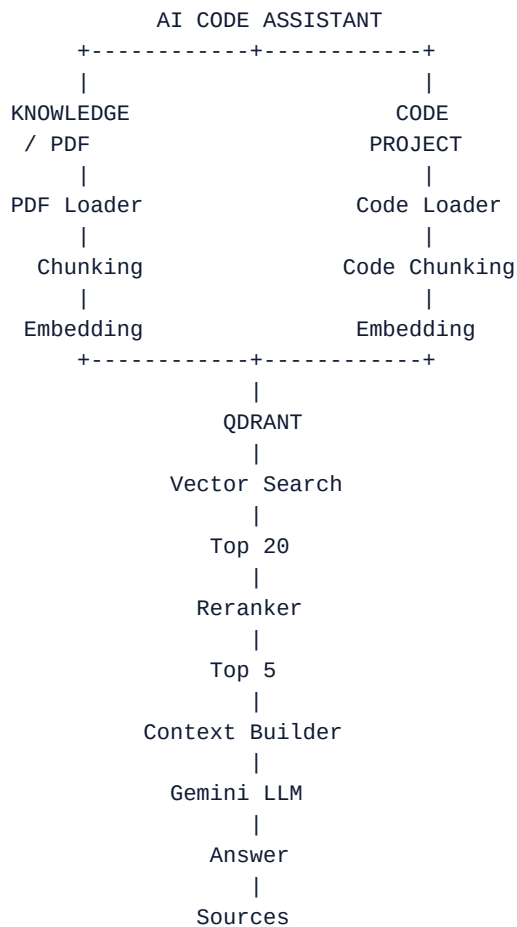
Retrieval is fast, broad candidate selection. Reranking is slower but more accurate relevance scoring.

What's the biggest next step?

The current system has a working PDF RAG core. The next major step is completing code ingestion and project-scoped code retrieval so the assistant can actually reason over a user's source code.

28 The Mental Model to Keep

Don't memorize every file — remember this shape instead.



Alongside it, PostgreSQL holds:

```

PostgreSQL
|-- Projects
|-- Chat Sessions
|-- Messages
+-- Long-term Memory

```

29 Official Project Checkpoint

Treat this as the source of truth going forward, rather than the older plan:

We have a functional Flask-based PDF RAG core with PDF extraction, chunking, hashing, duplicate-index prevention, embeddings, persistent Qdrant storage, vector retrieval, CrossEncoder reranking, Gemini generation, source metadata, and PostgreSQL conversation/project foundations. Codebase discovery/loading and project management have been started, but actual code chunking, code embedding/indexing, project-scoped retrieval, unified PDF+code retrieval, hybrid BM25 retrieval, active long-term memory, and advanced code understanding are still incomplete. The existing frontend is being set aside and will be rebuilt with Antigravity after the backend is stable.